CONTROL SYSTEMS

PID CONTROL

A practical study of feedback, tuning and real-world constraints

INTERNSHIP ASSIGNMENT Modern Monk Labs | Physical AI | Prepared by Parth Soni

PROJECT SNAPSHOT

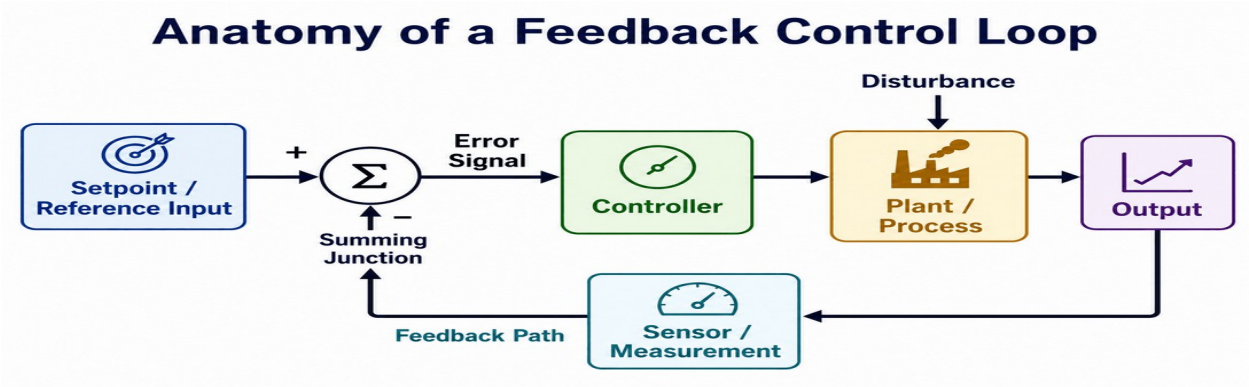
PLANT	TARGET	IMPLEMENTATION
1 kg mass–spring–damper	1.0 m position	Python • NumPy • Matplotlib

What this report demonstrates

A PID controller repeatedly compares a measured output with a target and adjusts the actuator command to reduce the error. This report moves from the feedback concept to the mathematical model, then uses experiment graphs to explain gain selection, actuator limits, sensor imperfections and tuning.

The feedback loop

SIGNAL PATH Setpoint → Error calculation → PID controller → Actuator / plant → Sensor → measured output returned to the controller



PID in one equation

CONTINUOUS FORM $u(t) = Kp \cdot e(t) + Ki \cdot \int e(\tau)d\tau + Kd \cdot de(t)/dt$ where $e(t) = r(t) - y(t)$

P responds to present error. I accumulates past error and can remove steady-state offset. D responds to the rate at which error changes and can reduce overshoot or oscillation. For this force–position plant, Kp is in N/m, Ki in N/(m·s), and Kd in N·s/m.

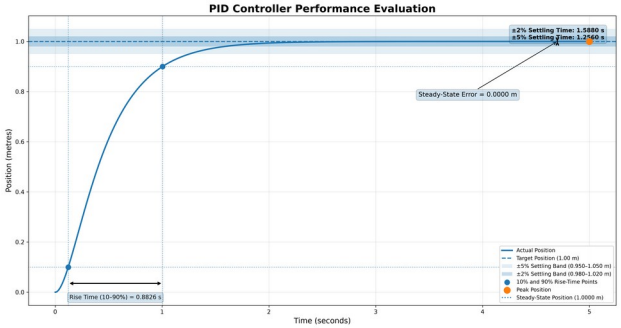
Test plant and performance criteria

The assigned plant has mass m=1 kg, damping c=10 N·s/m and spring constant k=20 N/m. Its equation is $m\cdot\ddot{x} + c\cdot\dot{x} + kx = u(t)$. At x=1 m and rest, the spring pulls with −20 N; the actuator must apply +20 N to hold the target.

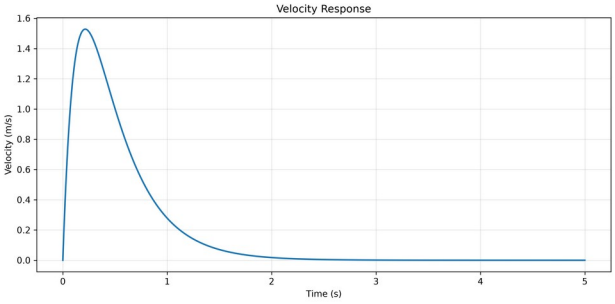
METRIC	ENGINEERING MEANING
Rise time	Time for the response to move from 10% to 90% of the step.
Overshoot	Peak amount above the target, expressed as a percentage.
Settling time	Time until the response remains inside the chosen tolerance band (typically ±2%).
Steady-state error	Difference between target and final measured position.

2. Open-loop response

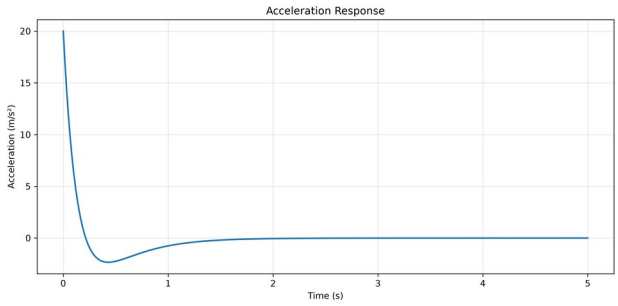
The open-loop experiment applies a constant input without using feedback to correct the output. It establishes the plant's natural response and validates the spring equilibrium relation.



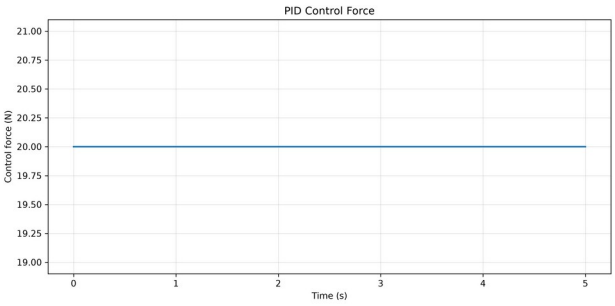
01 Open Loop | Position Response



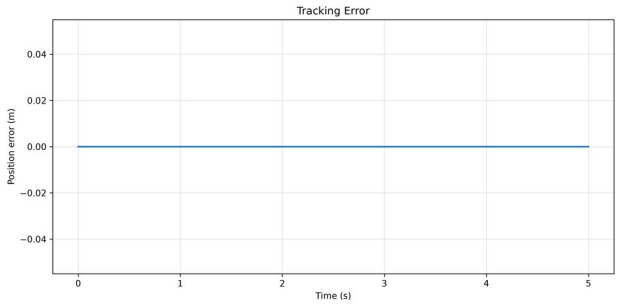
01 Open Loop | Velocity Response



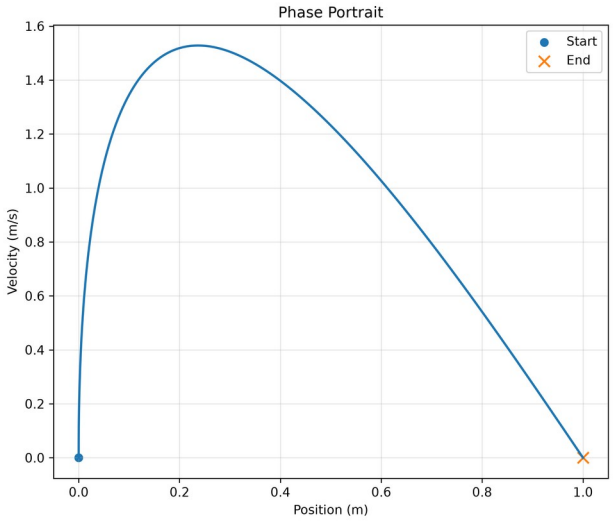
01 Open Loop | Acceleration Response



01 Open Loop | Control Force



01 Open Loop | Error Response

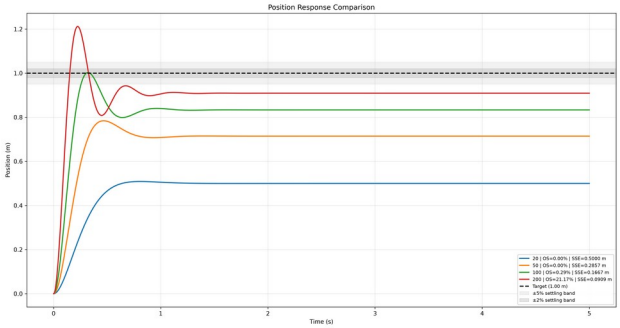


01 Open Loop | Phase Portrait

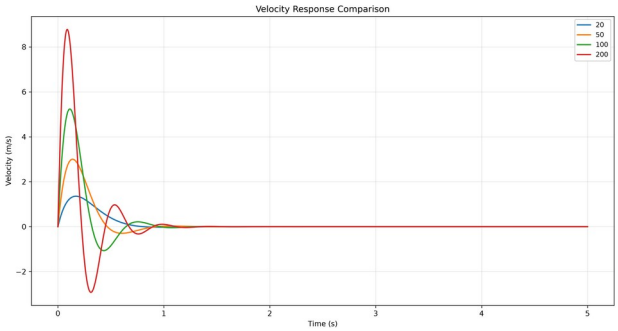
ENGINEERING READOUT At 20 N input, the static equilibrium is x=F/k=20/20=1 m. The position should approach 1 m while velocity and acceleration decay toward zero. Open loop reaches the target only because the applied force happens to match this plant's holding requirement; a disturbance or parameter change would not be corrected.

3. P-only control — gain sweep

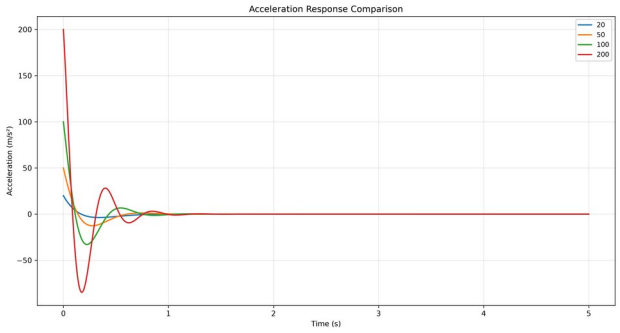
The proportional controller applies force in direct proportion to the current position error. Four gain values (Kp = 20, 50, 100 and 200) are compared.



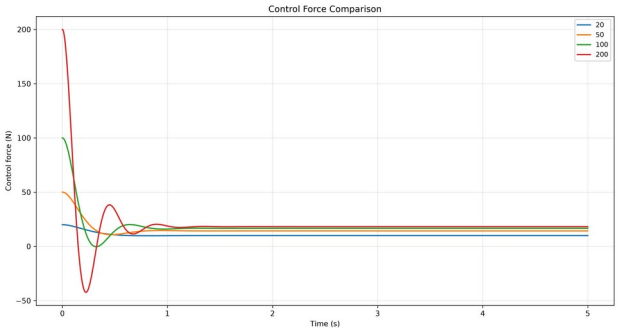
02 P Only | Position Comparison



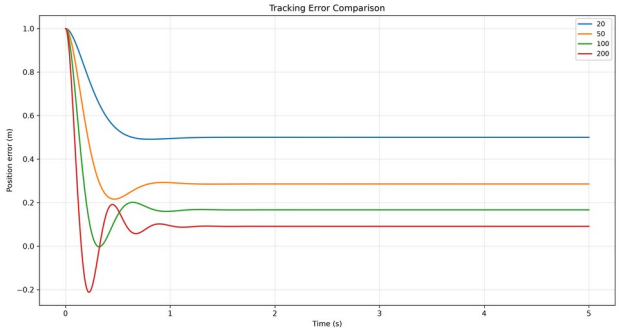
02 P Only | Velocity Comparison



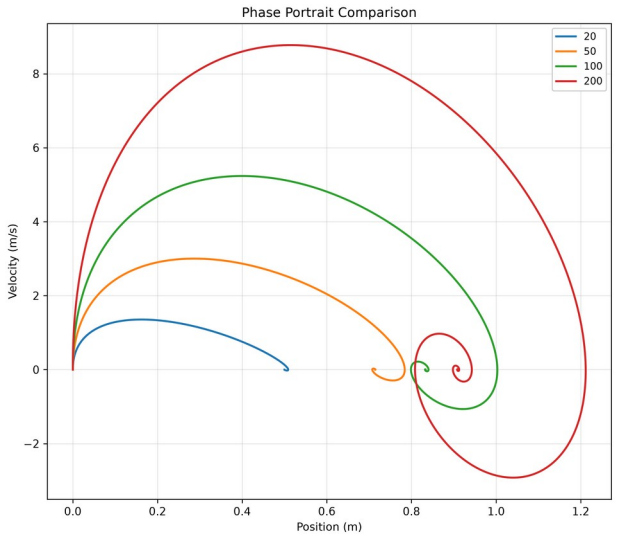
02 P Only | Acceleration Comparison



02 P Only | Control Force Comparison



02 P Only | Error Comparison

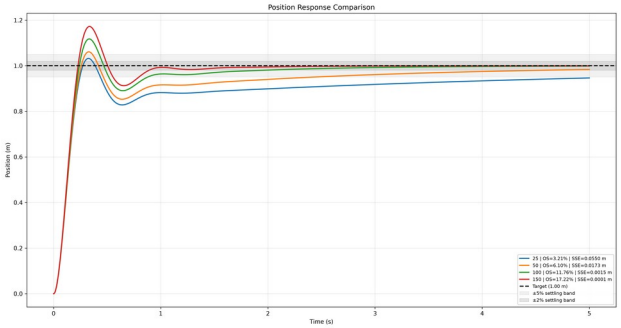


02 P Only | Phase Portrait Comparison

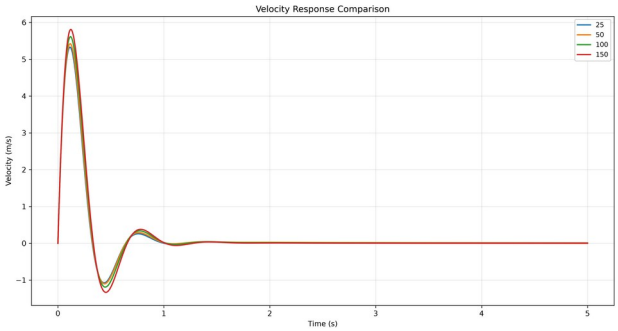
ENGINEERING READOUT For a spring-loaded plant, equilibrium requires Kp(r−x)=kx. Thus xss=Kp·r/(Kp+k) and SSE=r·k/(Kp+k). With r=1 m and k=20 N/m, predicted SSE values are 0.5000, 0.2857, 0.1667 and 0.0909 m respectively. Larger Kp reduces offset, but the supplied traces show stronger transient excursions at high gain.

4. PI control — integral action

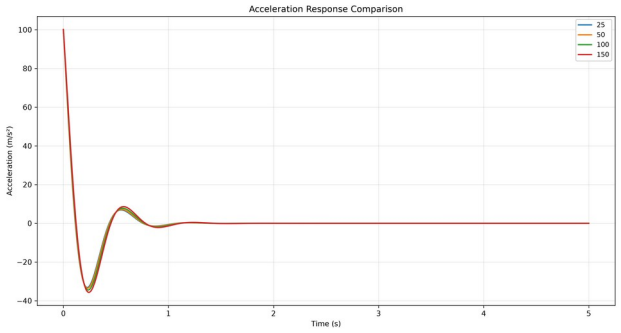
The PI controller combines present error with accumulated error. The integral term continues to build while a persistent offset remains, providing the extra force needed to balance the spring.



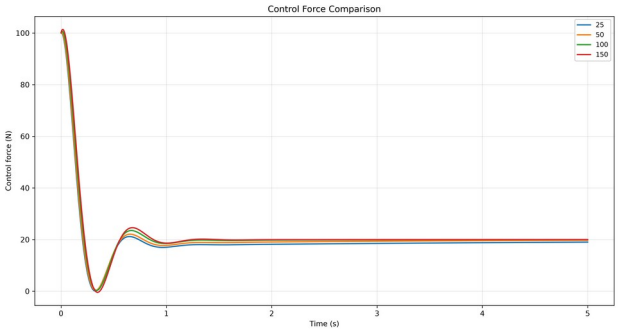
03 Pi Control | Position Comparison



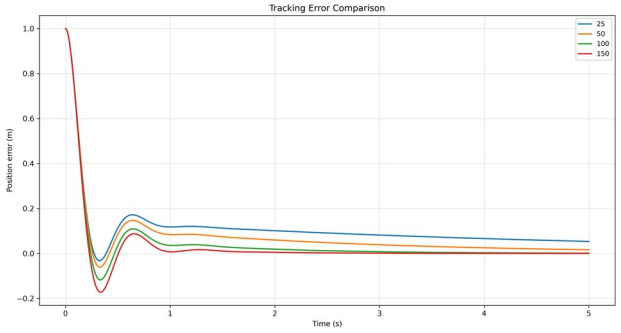
03 Pi Control | Velocity Comparison



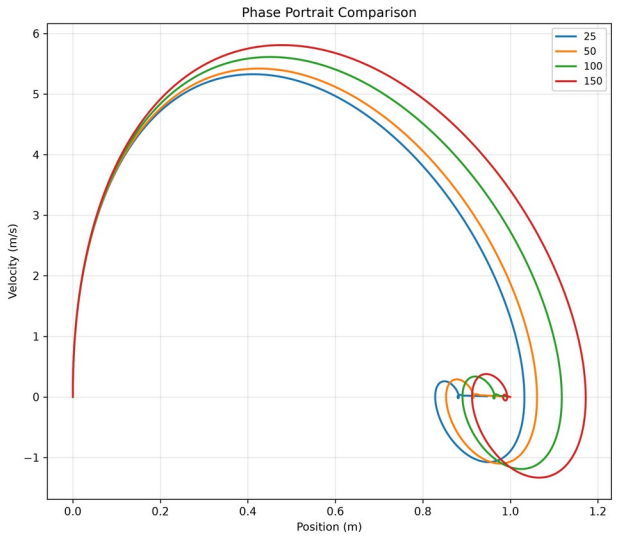
03 Pi Control | Acceleration Comparison



03 Pi Control | Control Force Comparison



03 Pi Control | Error Comparison

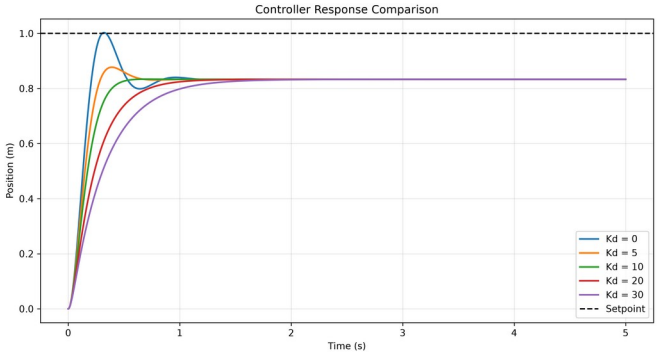


03 Pi Control | Phase Portrait Comparison

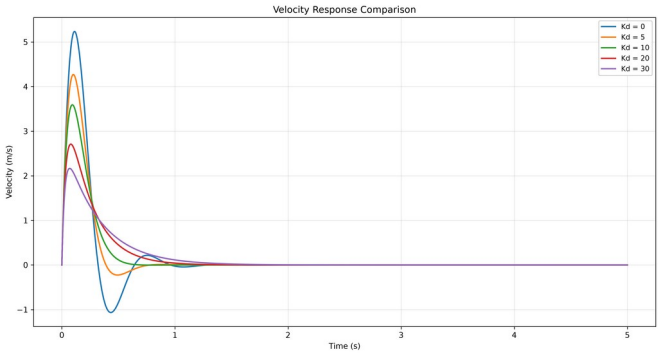
ENGINEERING READOUT Increasing Ki reduces final position error, but the integral contribution can continue driving the plant through the target. Choose Ki gradually: the objective is to remove offset without creating unacceptable overshoot, long settling time or windup.

5. PD control — derivative gain sweep

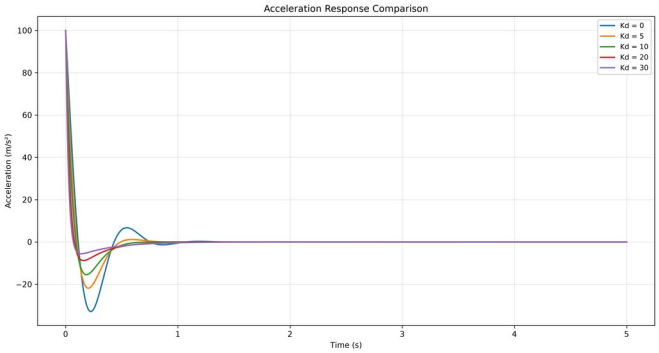
The PD controller uses proportional and derivative terms with Ki=0. The Kd=0 run is the P-only reference; larger Kd values add rate-sensitive correction.



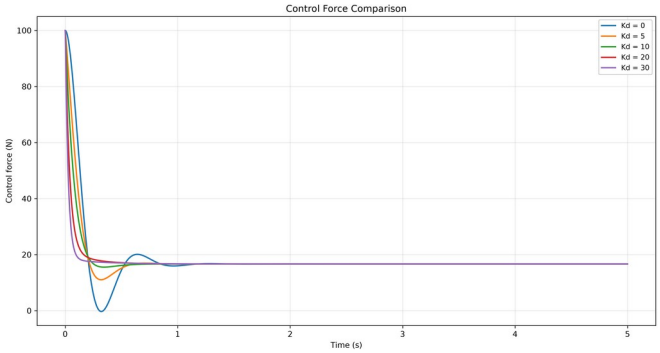
Kd | Position Response



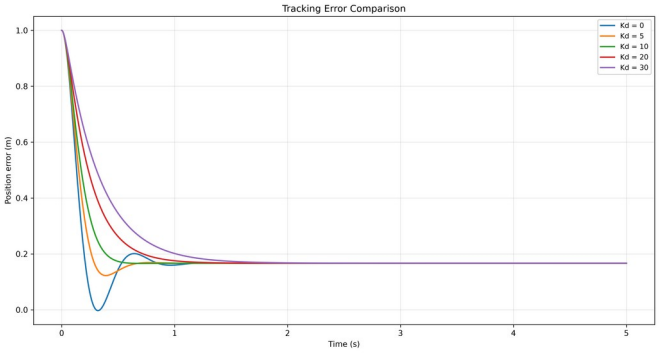
Kd | Velocity Response



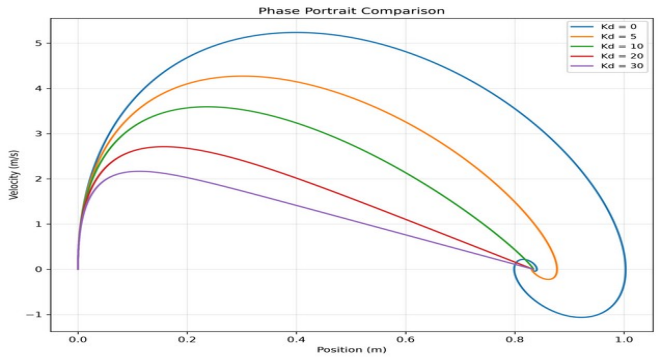
Kd | Acceleration Response



Kd | Control Force



Kd | Error Response



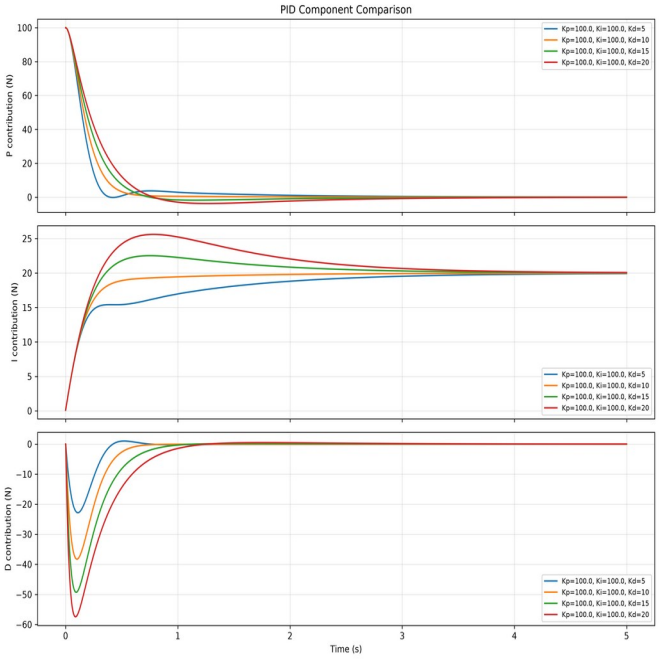
Kd | Phase Portrait

ENGINEERING READOUT Derivative action opposes rapid error changes and can reduce bouncing. Compare position and velocity for overshoot, acceleration for force reversals, and control force for actuator demand. Because Ki=0, PD generally retains the P-only steady-state offset on this spring-loaded plant.

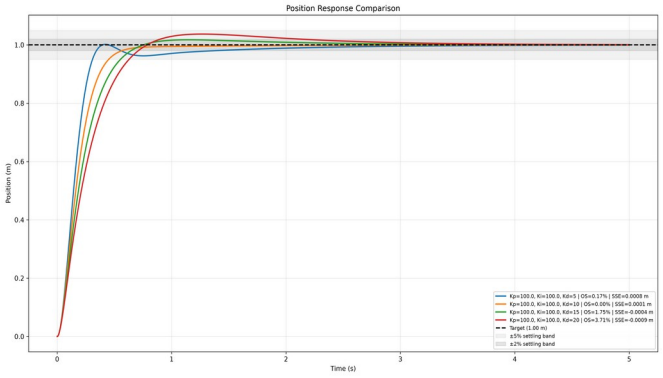
A larger Kd can reduce oscillatory motion but may slow the approach to the target and amplify sensor noise. Do not interpret a smoother position trace in isolation: check acceleration, force and error as well.
That is Rise Time and the slope of the error v/s time graph becomes gentle.

6. PID control — signal comparison

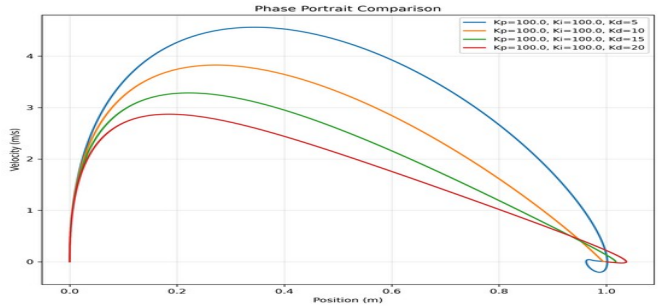
This page contains the PD controller comparison and the higher-derivative run signals. The same plant and proportional setting are used while derivative gain changes.



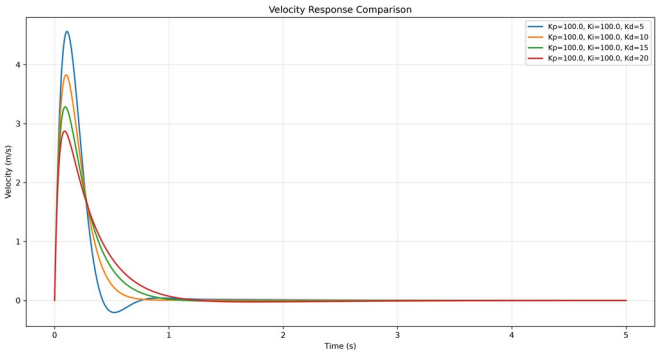
PID Components



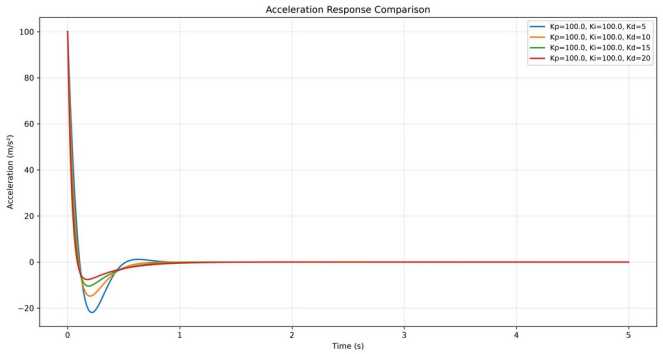
Position Response



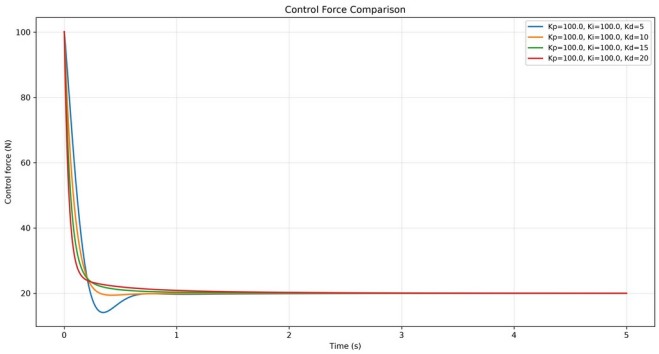
Phase Portrait



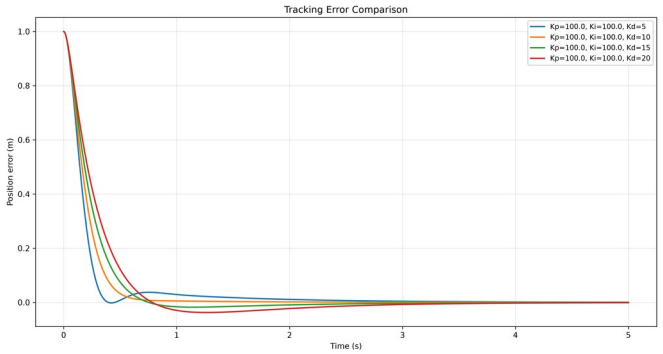
Velocity Response



Acceleration Response



Control Force

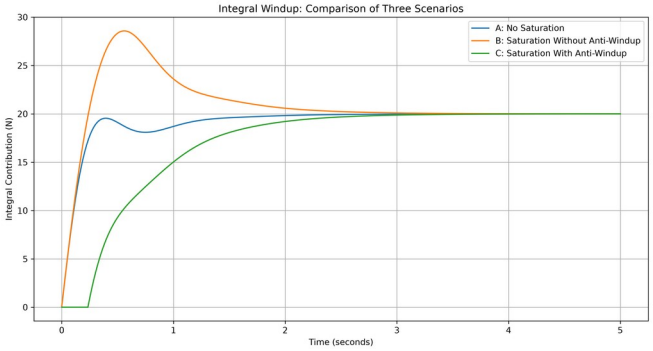


Error Response

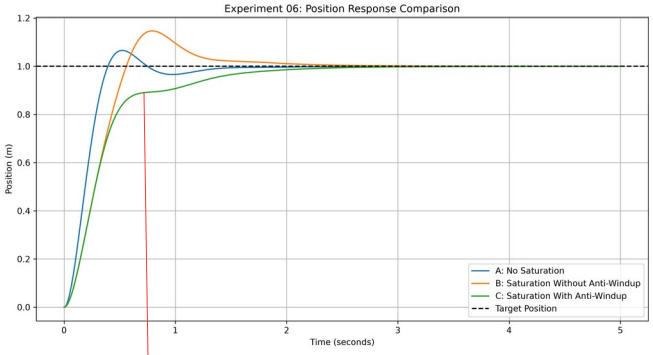
ENGINEERING READOUT A larger Kd can reduce oscillatory motion but may slow the approach to the target and amplify sensor noise. Do not interpret a smoother position trace in isolation: check acceleration, force and error as well.

7. Actuator saturation and integral windup

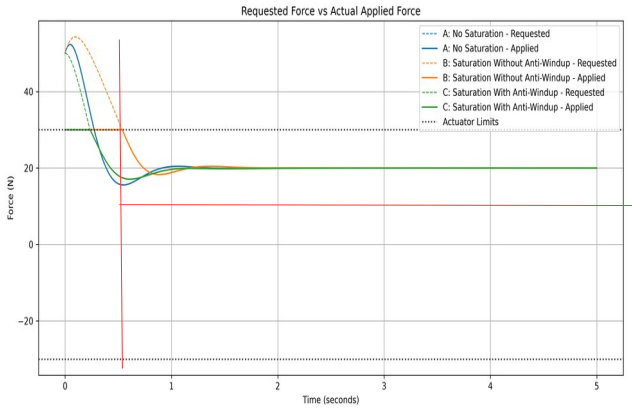
The actuator is limited to ±30 N. Requested controller force may exceed the limit, but the plant receives only the clipped applied force. Integral windup occurs when accumulated error continues growing while the actuator cannot provide the requested input.



Integral Contribution



Position Response



Control Force

We could probably close-in the Green Curve by just increasing the KI value, Try It Out !

But while using the Anti-windup it causes increase in the Settling Time

At this instant Green Curve (Anti-windup) is more close to the Equilibrium Force F = 20 where as Orange Curve (without Anti-windup) is still unwinding from the saturation

The Main benefit of applying Anti-windup is controlling the overshooting of the requested Force as visible in the Orange Curve which could put unnecessary on the Actuator and could lead to real life failure

ENGINEERING READOUT Conditional integration should stop the integral from growing only when the error pushes further into saturation. It should resume when the error helps the controller leave saturation. Anti-windup does not raise the ±30 N physical limit. Judge the fix using position response as well as force and integral plots.

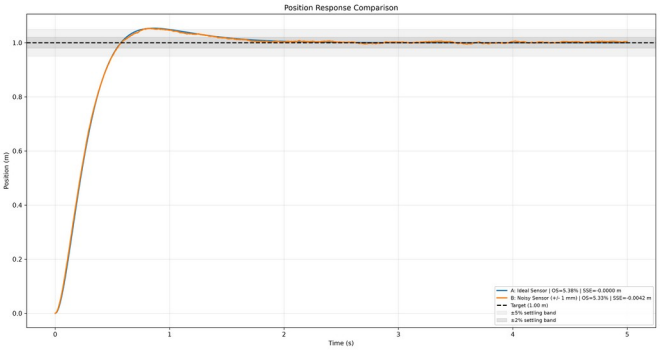
In case of Real Scenarios where the Actuators cannot be pushed to the maximum operating threshold all the time, applying the Concept of Actuator Saturation and Anti-Windup is much useful

Actuator Saturation: Applies a physical limitation to the PID Controller output to restrict any output greater than the specified threshold

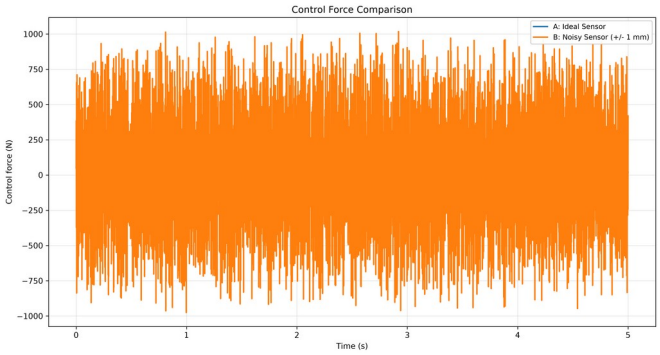
Anti-Windup: It Momentarily stops adding the new Integral gains as long as the PID Controller is operating above the threshold and meanwhile uses the previous Integral add-ups until it come back below the threshold levels

8. Position-sensor noise

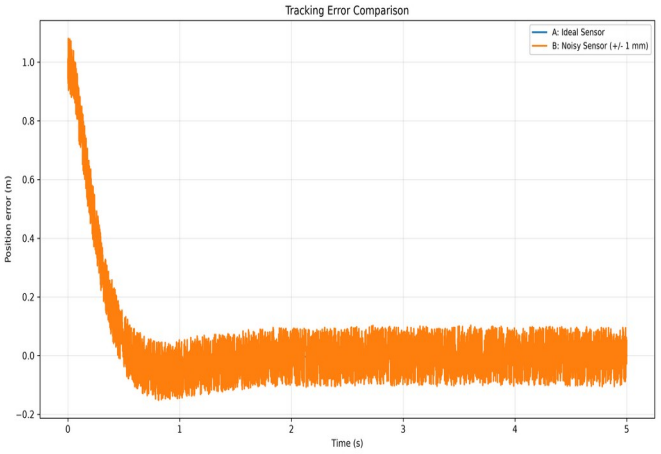
The sensor experiment compares actual plant position with noisy measurements and shows the effect on tracking error and force. The specified noise bound is ±0.1 m.



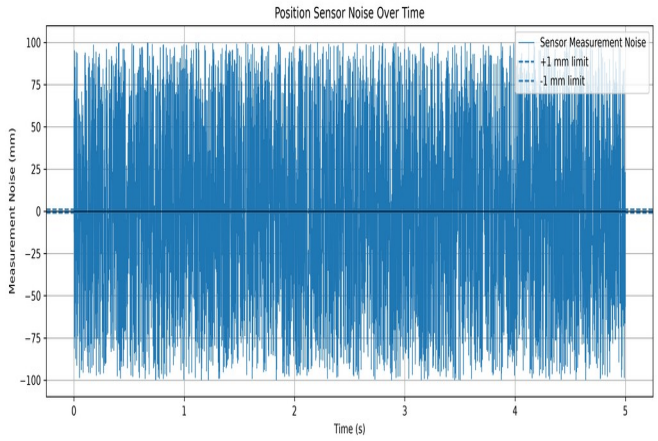
Actual Position v/s Noisy Sensor Positon



Control Force



Error Response

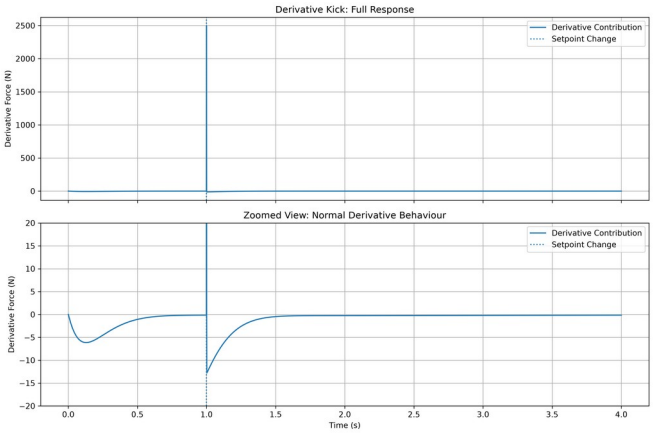


Phase Portrait

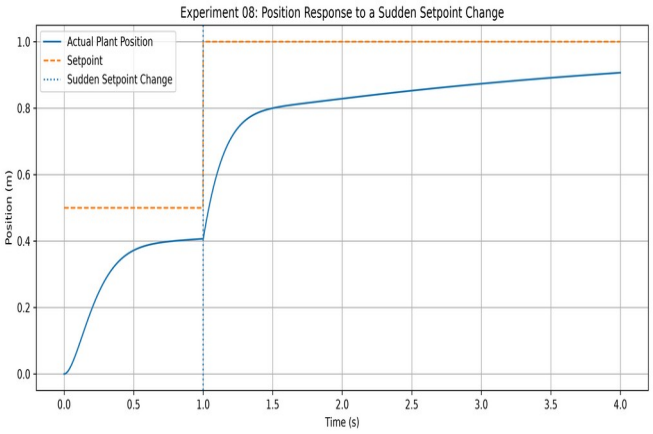
ENGINEERING READOUT Noise belongs in the measurement path, not in the plant's true state. Derivative action is especially sensitive because it differentiates sample-to-sample measurement changes. Use a fixed random seed for repeatable comparisons, and consider filtering or reducing Kd if force becomes noisy.

9. Derivative kick Slow Controller Updates

A sudden setpoint change produces a sharp error change. When D is calculated from error, this can create a large derivative command. The slow-update experiment which is after this separately examines the effect of holding the previous force between controller updates.



04 Pid Control | Position Comparison



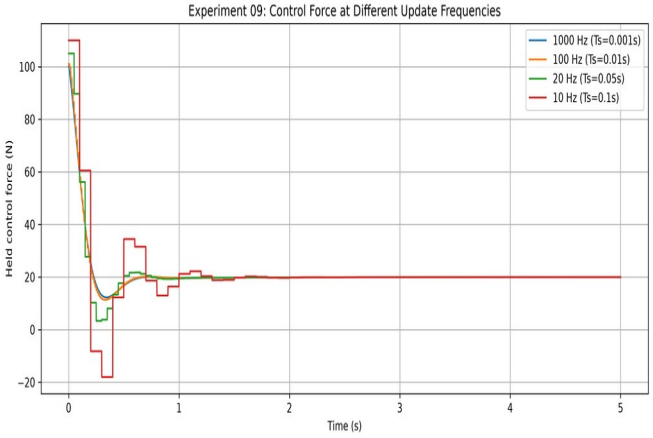
04 Pid Control | Velocity Comparison

The Algorithm is designed such a way that as soon as the Time > 1 second the previous Setpoint = 0.5 m is change to a new Setpoint = 1m due to which the Change in Error in the Derivative Term (de(t)/dt) spike which creates a Derivative Kick

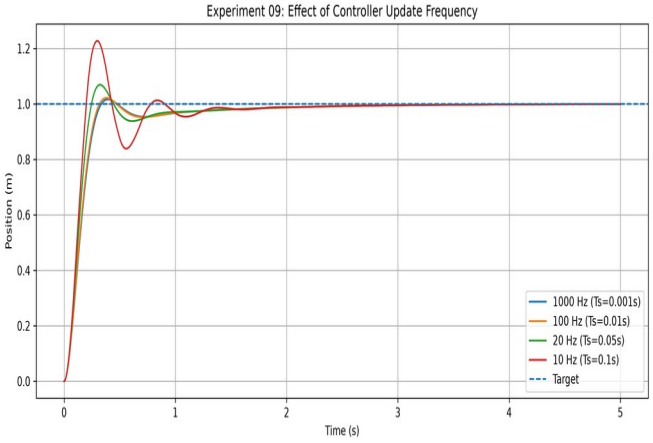
ENGINEERING READOUT For a 0.5 m setpoint step, Kd=5 & dt=0.001 s, the idealized derivative contribution (D) is 5×(0.5/0.001)=2,500 N. The first Iteration (step=0) guard an artificial startup derivative spike by setting previous_error = None; due this safety flag though initially the delta (e(t)) is high we don't get a Derivative kick initially .

10.Slow Controller Updates

For the Following Experiment we manipulated the Rate at which the PID Controller was giving output force (Feedback Force). But during the whole time the other state parameters like position (x) , velocity (v) , net acceleration of the system (a_net) was calculate with step size of 0.001s



Control Force – Time Response



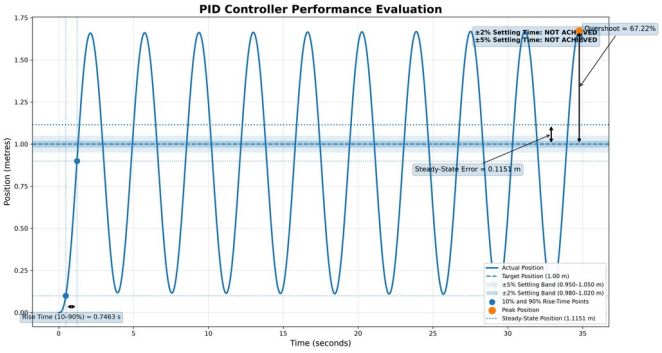
Position Response0

ENGINEERING READOUT For a lower Update Rate (frequency = 10Hz) of the PID Controller the system is jerky and cause a large overshoot as seen in the Position Response diagram. This type of purposefully causing a delay can help us analyze a Real world system to counter uncertainties

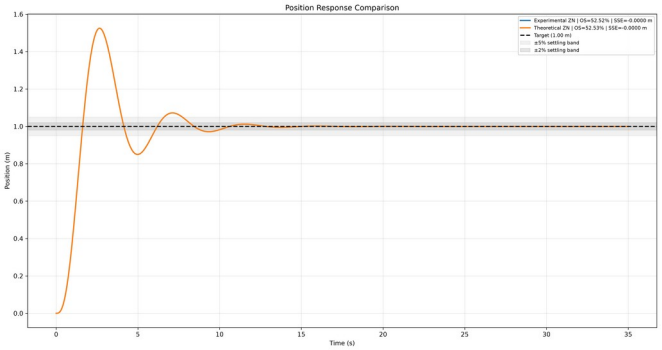
11. Ziegler–Nichols black-box tuning

The black-box experiment raises proportional gain until sustained oscillation, then records the ultimate gain Ku and oscillation period Tu. The resulting values provide a first PID tuning estimate.

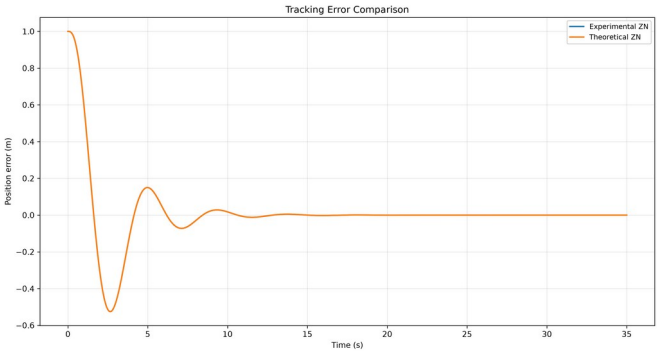
In this we try to find out the Perfect PID parameters of an Unknown Transfer Function (TF) whose mathematical model is not unknown to us. By just dealing with the inputs and the outputs of the TF we gradually increasing Gain Value until we reach a perfect oscillation . We note the particular Gain value also known as Ultimate Gain (Ku) and Time Period (Tu).Than with the Help of ZN table we calculate the Kp , Ki , Kd values



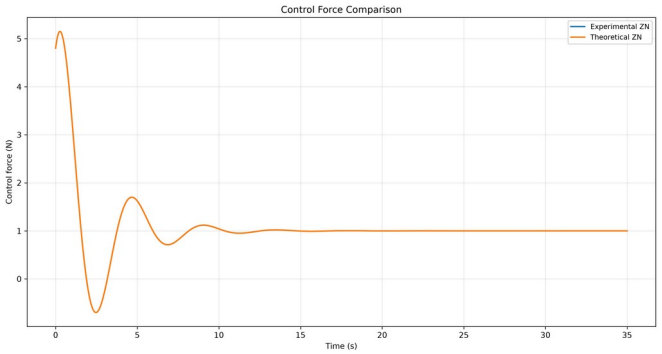
Position Response (Oscillation)



Experimental ZN v/s Theoretical ZN



06 Actuator Saturation | Control Force Comparison



06 Actuator Saturation | Integral Contribution

ENGINEERING READOUT For the reference plant G(s)=1/(s+1)^3 with unity feedback, Routh analysis gives Ku=8, ωu=√3 rad/s and Tu=2π/√3≈3.628 s. Classic PID estimates are Kp=4.8, Ti≈1.814 s, Td≈0.454 s, Ki≈2.646 and Kd≈2.177. Treat ZN as a starting point and refine it against the actual response.

Ziegler–Nichols (ZN) PID Tuning for $G(s) = \frac{1}{(s+1)^3}$

1 Plant and Setup

- Plant: $G(s) = \frac{1}{(s+1)^3}$
- Unity feedback
- Start with proportional control: $C(s) = K_p$

2 Closed-Loop Characteristic Equation

$$1 + K_p G(s) = 0$$

$$(s+1)^3 + K_p = 0$$

$$s^3 + 3s^2 + 3s + (1 + K_p) = 0$$

3 Routh–Hurwitz Analysis

s^3	1	3
s^2	3	$1 + K_p$
s^1	$\frac{8 - K_p}{3}$	0
s^0	$1 + K_p$	0

- Stability: $K_p < 8$
- Ultimate gain: $K_u = 8$

4 Ultimate Frequency and Period

At $K_p = K_u = 8$:

Auxiliary equation: $3s^2 + 9 = 0$

$$\omega_u = \sqrt{3} \text{ rad/s}$$

$$T_u = \frac{2\pi}{\omega_u} = \frac{2\pi}{\sqrt{3}} \approx 3.628 \text{ s}$$

5 ZN PID Tuning Rules (Classic)

Parameter	Formula
K_p	$0.6\,K_u$
T_i	$T_u / 2$
T_d	$T_u / 8$

6 Calculate Controller Gains

$$K_p = 0.6 \times 8 = 4.8$$

$$T_i = \frac{3.628}{2} \approx 1.814 \text{ s}$$

$$T_d = \frac{3.628}{8} \approx 0.454 \text{ s}$$

Convert to Parallel Form

$$K_i = \frac{K_p}{T_i} \approx 2.646$$

$$K_d = K_p T_d \approx 2.177$$

Final PID Controller (Initial ZN Tuning)

$K_p = 4.8$ | $K_i = 2.646$ | $K_d = 2.177$

Note: These ZN gains are a starting point. Refine them using the actual system response (e.g., to reduce overshoot or settling time).

14. Engineering conclusions & implementation

The experiments illustrate that PID tuning is a multi-objective engineering decision. A fast response may require large force and acceleration; low overshoot may increase rise or settling time; low steady-state error may require integral action; and sensor noise can make aggressive derivative action impractical.

Controller comparison — qualitative summary

CONTROLLER	STRENGTH	LIMITATION
P	Simple, fast correction	Finite steady-state error under spring load
PI	Removes persistent offset	Can overshoot; integral windup under saturation
PD	Improves damping / reduces bouncing	Does not remove P-only offset; D amplifies noise
PID	Balances tracking accuracy and transient damping	More parameters; needs careful tuning and practical safeguards

Reusable Python architecture

MODULE	INPUTS → OUTPUTS / RESPONSIBILITY
src/pid_controller.py	Inputs: Kp, Ki, Kd, setpoint, measured position and dt. Output: force command. Retains P/I/D terms and controller history; reset() clears state.
src/simulation.py	Inputs: plant, controller, duration, dt and optional position_sensor_obj. Output: time-series arrays for actual/measured position, velocity, acceleration, error, force and PID terms.
src/performance.py	Inputs: simulation result dictionary and setpoint. Outputs: rise time, overshoot, settling time, steady-state error and standardized plots.
plants / sensors	Plant modules implement physical dynamics; sensor module produces measurements and optional bounded noise.
experiments/	Each script selects gains and run conditions, calls shared simulation and performance functions, and writes figures to results/graphs/.

Real robot applications

Motor speed: encoder RPM feedback adjusts motor voltage. Drone attitude: IMU feedback adjusts motor commands. Robot arm: joint encoders provide angle feedback for torque or velocity control. Line follower: sensor offset drives steering correction. Vehicle cruise control: speed feedback adjusts throttle or braking.

Common PID myths

1. “PID always removes offset” — P-only may retain offset; integral action can remove it when actuator authority permits. 2. “More Kp is always better” — high gain can cause overshoot or instability. 3. “D predicts the future” — it reacts to the rate of change. 4. “Anti-windup removes actuator limits” — it manages integral accumulation, not the hardware limit. 5. “One gain set works everywhere” — tuning depends on plant, sampling, load and actuator constraints.

Reproducibility and AI-use statement

The supplied results archive contains 78 graph files covering open loop, P, PI, PD, PID, saturation, sensor noise, derivative kick, slow update and Ziegler–Nichols. The project uses Python, NumPy and Matplotlib, with PID logic implemented directly. AI assistance was used to organize explanations and review documentation; final numerical values should be verified by rerunning the code.

FINAL ENGINEERING TAKEAWAY Tune against the complete response: position accuracy, transient behaviour, actuator effort, sampling rate and measurement quality. No single gain value is best independently of the plant and operating conditions.